ISTANBUL TECHNICAL UNIVERSITY
FACULTY OF COMPUTER AND
INFORMATICS

TinyNPU

Graduation Project Final Report

Fırat Kızılboğa
150200115

Department: Computer Engineering
Division: Computer Engineering

Advisor: Assoc. Prof. Dr. Ayse Yilmazer

March 2026

Statement of Authenticity

I/we hereby declare that in this study

1. all the content influenced from external references are cited clearly and in detail,
2. and all the remaining sections, especially the theoretical studies and implemented software/hardware that constitute the fundamental essence of this study is originated by my/our individual authenticity.

İstanbul, Mart 2026

Fırat Kızılboğa

Acknowledgments

I would like to thank my advisor, Assoc. Prof. Dr. Ayse Yilmazer, for her guidance throughout this project. I also thank the people whose open-source tooling and research made it possible to build, analyze, and validate the TinyNPU hardware and software stack.

TinyNPU

(SUMMARY)

TinyNPU is a compact accelerator project that evolved into a complete hardware/-software stack. The project now includes a parameterized systolic-array-based neural processing unit, a PyTorch-first segmented compiler and runtime, a quantization-aware model preparation flow, RTL and host-side verification infrastructure, and performance and synthesis characterization tooling. The goal of the project is not merely to design a datapath, but to make that datapath usable from a realistic machine-learning workflow and measurable in a disciplined way.

The hardware architecture is organized around an $N \times N$ output-stationary systolic array, a unified buffer, an instruction memory, streamer logic that feeds the array, and a post-processing unit that applies bias, rescaling, activation, and packed writeback. The software stack traces PyTorch models, partitions them into accelerator segments and host operations, compiles supported segments into TinyNPU instruction and buffer images, and executes the resulting plan on either host emulation or RTL simulation. Unsupported work is never hidden; it appears explicitly as host-side steps in the execution plan.

One of the main lessons of the project is that the useful unit of design is the whole stack, not the RTL alone. Compiler-visible tensor semantics, physical packing format, host/NPU boundaries, verification points, and repeated-run runtime behavior all affect whether the accelerator is practical to use. For this reason, the project includes explicit host-op dispatch, compiler-owned expected tensors for verification, repeated-run sessions with persistent state, and benchmark accounting that separates accelerator compute cycles from runtime and protocol overhead.

The current experimental evidence is intentionally narrow but concrete. The implemented stack has been validated end-to-end on a trained MNIST model, and a controlled multi-tile matrix multiplication benchmark has been used to study compute speedup and integration overhead. In addition, a reproducible Yosys-based synthesis flow provides rough resource estimates for the current design. The report therefore presents TinyNPU not as a finished industrial inference platform, but as a working accelerator research system whose main value lies in understanding architecture, compilation, packing, and workload fit together.

TinyNPU

(ÖZET)

TinyNPU, özel bir nöral işlemci birimi ile onu kullanılabilir hale getiren derleyici, çalışma zamanı, niceleme akışı ve benzetim tabanlı doğrulama ortamını birlikte geliştirmeyi amaçlayan bir projedir. Projenin hedefi yalnızca küçük bir hızlandırıcı çekirdeği tasarlamak değil, bu çekirdeği gerçek bir makine öğrenmesi iş akışında kullanılabilir hale getirmektir. Bu nedenle proje, PyTorch tabanlı model hazırlama sürecini destekleyen, nicelemeli modelleri hızlandırıcı segmentlerine dönüştüren, desteklenmeyen işlemleri ana işlemci üzerinde çalıştıran ve sonuçları yazılım referanslarıyla doğrulayan bütünleşik bir donanım-yazılım yapısı içermektedir.

Donanım tarafında TinyNPU, birleşik tampon bellek ve komut belleği ile beslenen sistolik dizi tabanlı bir hesaplama çekirdeği etrafında kurulmuştur. Girdi aktivasyonları ve ağırlıklar birleşik tampondan akıtıcı birimlere, oradan da sistolik diziye iletilir. Hesaplama sonucunda oluşan değerler son işleme biriminden geçirilerek paketlenmiş biçimde tekrar belleğe yazılır. Bu yapı, yoğun nicelemeli matris işlemleri için verimli bir veri yolu sağlarken, dışarıdan görülen programlama modelini de yönetilebilir düzeyde tutmaktadır. Yazılım tarafı ise yüksek seviyeli model işlemlerini hızlandırıcı segmentleri ve gerekirse ana işlemci işlemlerini birlikte içeren bir yürütme planına dönüştürmektedir.

Bu projedeki temel ilerleme, izole donanım deneylerinden uçtan uca çalışan bir sisteme geçilmesi olmuştur. Eğitilmiş MNIST modeli dahil gerçek iş yükleri yazılım ve donanım yığını üzerinden çalıştırılabilmektedir. Ayrıca, hızlandırıcının gerçek hesaplama çevrimlerini entegrasyon ek yüklerinden ayıran bir ölçüm altyapısı geliştirilmiştir. Böylece çekirdeğin kendi verimliliği ile yazılım, paketleme ve arayüz maliyetleri ayrı ayrı değerlendirilebilmek Tekrarlı çıkarım senaryolarında gereksiz durum yüklemelerini azaltmak için oturum tabanlı çalışma ve ağırlık kalıcılığı desteği de eklenmiştir.

Projenin mevcut durumu, donanımın anlamlı hesaplama yaptığı, ancak sistem seviyesindeki davranışın derleyici bölütlemesi, bellek yerleşimi ve çalışma zamanı düzenlemesine güçlü biçimde bağlı olduğunu göstermektedir. Bu nedenle proje, "derleyiciyi var etmek" aşamasını aşmış; bunun yerine derleyici ve çalışma zamanının karmaşıklığını yönetmek, gereksiz çevrim maliyetlerini azaltmak ve hangi iş yüklerinin bu mimariye uygun olduğunu anlamak gibi daha olgun tasarım sorularına yönelmiştir. Bu rapor, ortaya çıkan sistemi, temel tasarım kararlarını ve işlevsellik ile performans değerlendirmesinde kullanılan yöntemi özetlemektedir.

Contents

1 Introduction and Project Summary **1**
1.1 Background and Motivation 1
1.2 Problem Definition 1
1.3 Project Scope and Approach 1

2 Comparative Literature Survey **3**
2.1 Mixed-Precision Quantization 3
2.2 High-Performance Matrix Multiplication 3
2.3 Existing Accelerator Architectures 3

3 Developed Approach and System Model **5**
3.1 Data Model 5
3.2 Structural Model 5
3.3 Dynamic Model 10
3.3.1 Execution Phases 11
3.3.2 Current Scope Boundaries 13

4 Experimental Setup and Design **14**

5 Comparative Evaluation and Discussion **15**

6 Conclusion and Future Work **17**

1 Introduction and Project Summary

1.1 Background and Motivation

Modern deep neural networks demand substantial compute and memory bandwidth even at inference time. This has made specialized accelerator design a central topic both for datacenter-scale systems and for smaller edge-oriented deployments [6, 4]. Quantization is one of the main techniques that makes such deployment practical, because it reduces memory traffic and allows arithmetic throughput to increase when narrow integer types are used instead of floating-point data.

However, quantization is not a single fixed target. Mixed-precision work has shown that different layers or kernels inside the same model can tolerate different bit-widths [7]. This creates a hardware opportunity and a software problem at the same time. Hardware should benefit from narrower arithmetic when it is safe to do so, but the software stack must still prepare the model, preserve numerical intent, and decide where accelerator execution ends and host execution begins.

1.2 Problem Definition

The main problem addressed by TinyNPU is therefore not only how to build a small systolic-array accelerator, but how to make mixed-precision quantized computation usable from a realistic model flow. Fixed-precision accelerators leave throughput on the table when different parts of a model have different precision needs. At the other end of the spectrum, CPU-centric approaches based on ISA extensions or soft SIMD can support low-bit computation, but they still pay substantial software-side cost for packing, unpacking, and orchestrating the data movement around those operations [3, 1, 2].

This project treats the problem as a co-design task. The hardware must expose a regular datapath that benefits from packed low-precision arithmetic. The compiler and runtime must make tensor layout, quantization boundaries, and host fallbacks explicit rather than implicit. The result should be a system that can be measured honestly: accelerator compute, runtime overhead, and host-side work should not be conflated.

1.3 Project Scope and Approach

TinyNPU approaches this problem with four coupled components:

1. **A parameterized mixed-precision accelerator:** TinyNPU is centered around an output-stationary $N \times N$ systolic array, a unified buffer, streamer logic that feeds the array, and a post-processing unit that performs bias addition, rescaling, activation, saturation, and packed writeback.

2. **Packed integer computation:** The processing elements support packed INT16, INT8, and INT4 execution modes. Throughput therefore scales with precision reduction, not by changing the global architecture, but by packing more useful work into the same datapath.

3. **A segmented compiler/runtime:** The software flow traces supported PyTorch models, partitions them into accelerator segments and explicit host-side operations, lowers legal accelerator work into instruction and buffer images, and executes the resulting plan on either host emulation or RTL simulation.

4. **Verification and measurement infrastructure:** The project includes compiler-owned expected tensors, runtime verification points, repeated-run sessions, and benchmark accounting that separates accelerator compute from protocol and software overhead.

The current project state should be read with that scope in mind. TinyNPU is already capable of running a trained MNIST model through the full compiler/runtime stack and of benchmarking a controlled multi-tile matrix multiplication workload. At the same time, workload coverage is still narrow, and the current results are most useful for understanding architecture, packing, compiler boundaries, and runtime cost rather than for claiming broad deployment readiness.

The rest of this report follows that logic. The next section positions TinyNPU relative to relevant literature in mixed-precision quantization, matrix multiplication, and accelerator design. The system model section then explains the implemented hardware and software stack in detail, followed by the experimental method, the current results, and the most important next steps.

2 Comparative Literature Survey

2.1 Mixed-Precision Quantization

Mixed-precision quantization work is the clearest algorithmic motivation for TinyNPU. Wu et al. formulate layer-wise quantization as a search problem and show that different layers can be assigned different bit-widths without treating the whole model uniformly [7]. This is the key observation behind a variable-precision accelerator: if model layers do not all need the same precision, a fixed-precision datapath cannot exploit the full efficiency opportunity.

Quantization is also relevant as a compiler concern rather than only a hardware concern. SmoothQuant is an example of pushing difficult numerical adjustments into an offline transformation so that runtime execution becomes simpler and more hardware-friendly [8]. TinyNPU follows the same general philosophy even though it does not implement the full SmoothQuant method itself: model preparation, fused rescaling, and compiler-ready conversion are treated as software responsibilities that simplify the accelerator contract.

2.2 High-Performance Matrix Multiplication

The mathematical core of TinyNPU is still matrix-style computation, so the project is strongly informed by classical GEMM literature. Goto and van de Geijn show that high-performance matrix multiplication depends not only on arithmetic throughput, but on careful tiling and data movement through the memory hierarchy [5]. The same principle appears in TinyNPU at a smaller scale. When matrices exceed the dimensions of the systolic array, the compiler and runtime must tile along the M, N, and K dimensions while preserving the data reuse pattern expected by the array and its post-processing path.

2.3 Existing Accelerator Architectures

On the accelerator side, Google's TPU remains the canonical reference for systolic-array-style matrix acceleration [6]. Gemmini is especially relevant because it demonstrates how a systolic array, programming model, and system context can be treated as one full-stack research object rather than as an isolated RTL block [4]. That full-stack perspective is one of the strongest methodological reference points for TinyNPU.

An earlier educational TinyTPU implementation was useful as a starting point for understanding systolic-array organization, but it was not a good endpoint for this project. Extending it toward mixed precision, tiled execution, and cleaner post-processing exposed the limits of treating the datapath as the whole problem. TinyNPU therefore diverged into a custom implementation with a different software contract, explicit

host/accelerator boundaries, and a compiler/runtime that owns packing and verification rather than leaving those concerns informal.

Taken together, these references define the design space in which TinyNPU sits. It is not trying to outscale large industrial accelerators. Its niche is a compact mixed-precision accelerator system in which the hardware, packing model, compiler/runtime, and measurement methodology can all be studied together in one working platform.

3 Developed Approach and System Model

The developed system consists of four tightly coupled parts: the RTL accelerator, the model preparation and quantization flow, the segmented JIT compiler/runtime, and the verification and benchmarking infrastructure. These pieces were built together because no single layer is meaningful in isolation. The hardware cannot be evaluated honestly without a realistic execution path, and the compiler cannot be trusted without RTL-backed validation.

3.1 Data Model

The core data model is built around explicit tensors, explicit quantization metadata, and explicit execution-plan artifacts. In the segmented compiler flow, each tensor is represented by a typed specification carrying shape, data type, tensor kind, optional constant data, verification intent, and metadata such as quantization parameters. This is important because TinyNPU does not operate on generic floating-point tensors at runtime. The compiler and runtime need to know exactly when a tensor is a floating host value, when it is a signed quantized value, and when it has already been converted into a packed accelerator-facing representation.

The next major data structure is the execution plan. Rather than generating a monolithic "program" for the whole model, the compiler builds an `ExecutionPlan` whose steps are typed as `NpuSegment`, `HostOp`, or `VerifyTensor`. `NpuSegment` groups together accelerator-supported matrix-style work. `HostOp` keeps unsupported or intentionally host-side work explicit. `VerifyTensor` allows compiler-owned expected tensors to be checked at runtime. This split is central to the project because it makes boundaries and fallback semantics visible instead of implicit.

Physical layout is also part of the data model. The current backend uses role-specific packings for tensors that feed the array horizontally, tensors that feed it vertically, output buffers that match the writeback layout, and packed bias vectors. These layouts are backend details rather than semantic tensor meanings, but they matter because the compiler must map logical tensors onto physical storage forms that match the datapath. A large part of the compiler work came from keeping this physical reality explicit without allowing it to leak upward and distort the logical semantics of the graph.

3.2 Structural Model

Structurally, TinyNPU is organized around a control path and a compute path. The control path is responsible for instruction orchestration and host service. In the current Runtime V2 path, data staging is primarily handled through the shared-SRAM host window, while MMIO commands remain the control path for launch/status and compatibility fallback. The compute path consists of the unified buffer, streamer logic, a parameterized $N \times N$ output-stationary systolic array, and the post-processing unit.

During active computation, data is read from the unified buffer, transformed into the orientation expected by the array, consumed by the processing elements, and then drained through the post-processing unit before being written back in packed form.

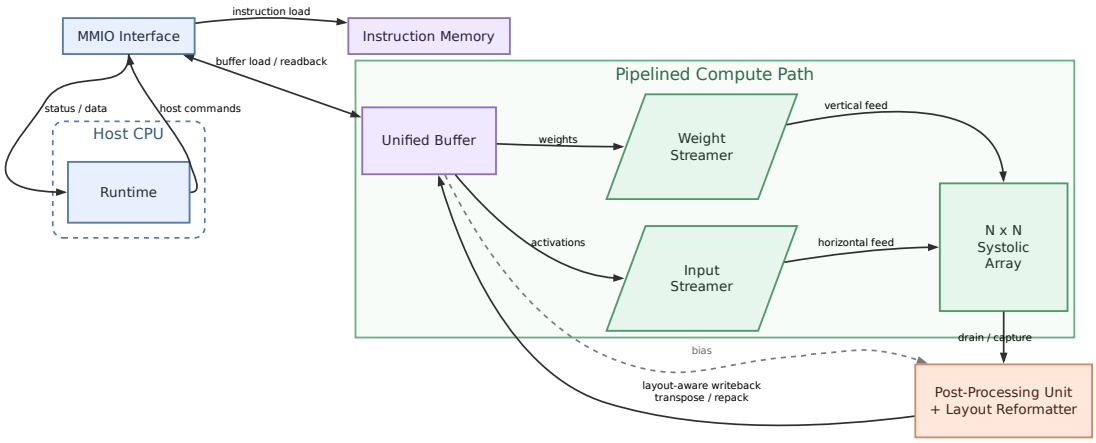

Figure 3.1: Top-level TinyNPU hardware architecture.

Figure 3.1 shows the top-level organization used throughout this report. The host-side runtime communicates with the accelerator through the MMIO interface, which is responsible for instruction loading and buffer access. Inside the accelerator, the unified buffer, input streamer, weight streamer, and systolic array form the main pipelined compute path. The post-processing unit is shown separately because it becomes active during the later drain and writeback phase rather than during the main feed phase of the array. This distinction is important for explaining both data movement and measured execution cost.

The processing element itself is also structurally important to the project. Each PE supports packed low-precision arithmetic rather than only a single scalar multiply. In the current implementation, a PE can perform four INT4 products, two INT8 products, or one INT16 product in a step, and accumulates the resulting partial sum in stationary form. The array therefore gains speed not only from spatial parallelism across rows and columns, but also from packed arithmetic within each PE. This is one of the main reasons the accelerator can look strong on dense quantized kernels even though the surrounding system overhead is still significant.

Packing follows the same hardware structure. Activation-side data is packed to match the horizontal feed direction, weight-side data is packed to match the vertical feed direction, output storage is packed to match the writeback layout, and bias vectors are packed in wide words that match the post-processing path. This role-specific packing is not a cosmetic implementation detail. It is part of the architecture contract, and it strongly influences the compiler and runtime because legal execution depends on matching physical layout to the actual dataflow through the array and post-processing

logic.

On the software side, the key structural idea is segmented execution. Instead of forcing the entire model into a single monolithic accelerator program, the compiler identifies supported regions and emits them as accelerator segments, while unsupported or intentionally host-resident work remains as host operations. This representation is simpler than a full whole-graph hardware-only compiler, but it makes the cost of boundaries, packing, and readback visible. The runtime then executes the plan step by step, coordinating accelerator launches and host computation as needed.

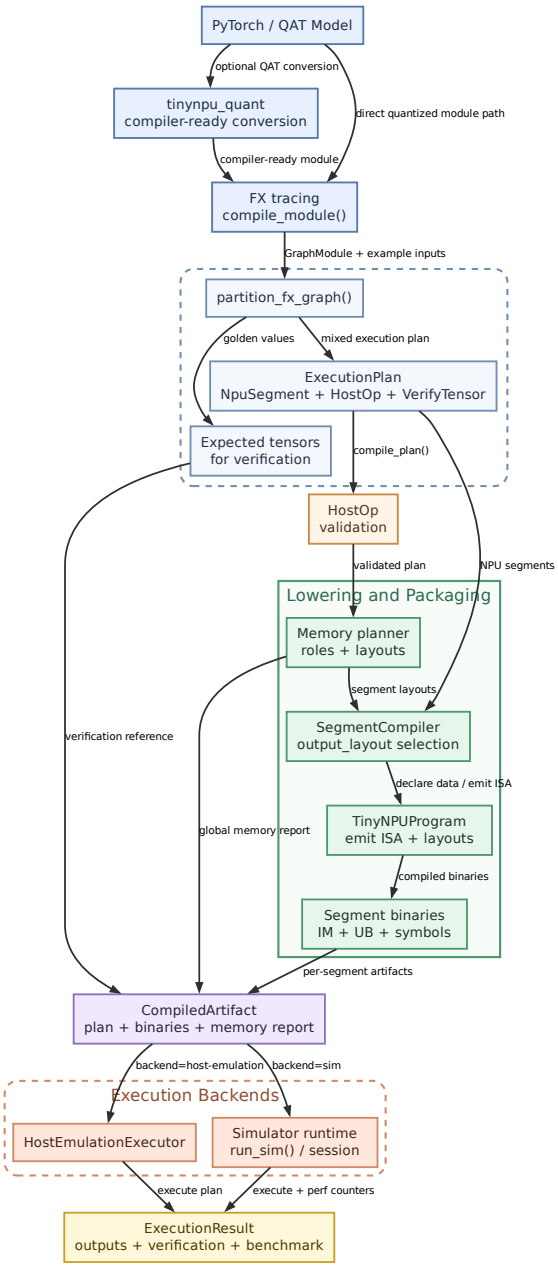

Figure 3.2: Compiler and runtime flow used by the TinyNPU JIT path.

Figure 3.2 summarizes the software flow implemented in the repository. A model enters either directly as a supported quantized PyTorch module or through a compiler-ready conversion path built around `tinynpu_quant`. The JIT frontend traces the model with FX, partitions it into a mixed execution plan, validates host operations, and lowers each accelerator segment through the memory planner and `SegmentCompiler`. The resulting `CompiledArtifact` contains the execution plan, per-segment IM/UB binaries, expected tensors for verification, and memory-layout information required by the runtime. In Runtime V2, the generated artifact is intentionally static-data-heavy: emitted C files carry program tables and constants, while shared runtime code executes those tables through a fixed interpreter-style loop.

Another important structural choice is that benchmarking is built into the execution path. The runtime can collect hard RTL cycle counts for accelerator execution windows and keep separate accounting for software-side overhead such as buffer loading and packing. This allows the project to reason about compute efficiency without confusing it with prototype integration overhead.

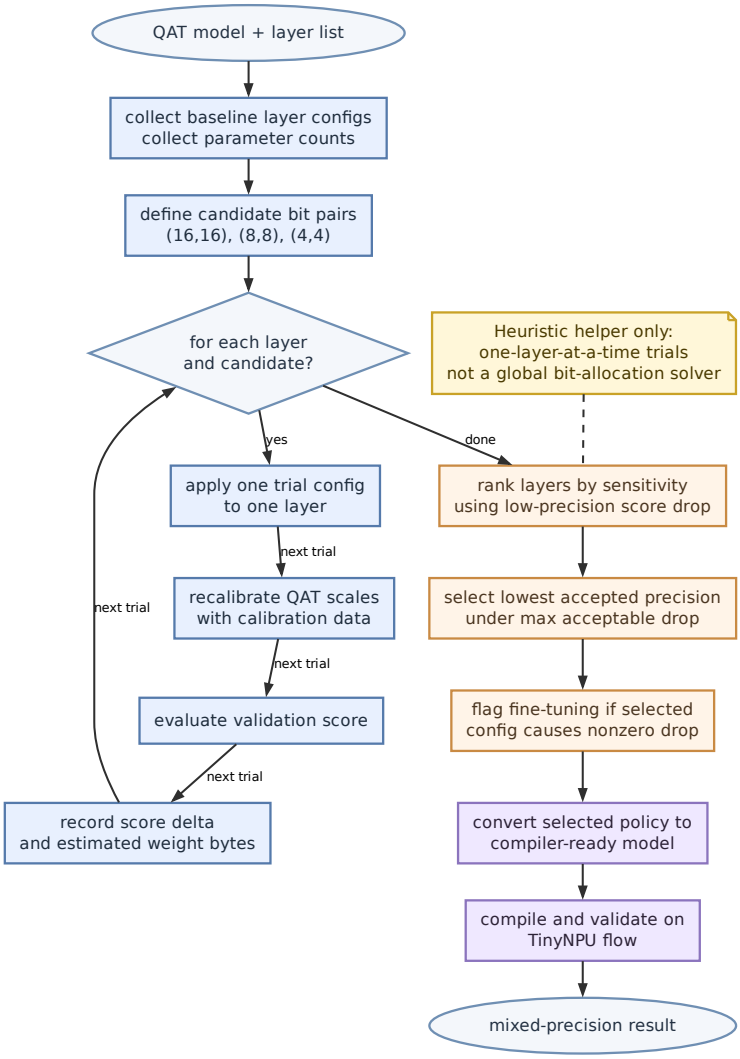

Figure 3.3: Heuristic mixed-precision selection flow used to obtain compiler-ready model variants.

Figure 3.3 shows the mixed-precision selection helper currently used in the project. It is important to state its scope precisely. The project is not attempting to solve the general bit-allocation problem or to claim an optimal mixed-precision policy. Instead, it implements a simple one-layer-at-a-time sensitivity workflow that is sufficient to generate candidate mixed-precision configurations, recalibrate them, and feed the selected result into the compiler-ready conversion path. This keeps the project focused on obtaining usable results in the TinyNPU flow rather than overclaiming a global mixed-precision optimization method.

3.3 Dynamic Model

Dynamically, the system operates as a mixed host-accelerator execution pipeline. A prepared model is first converted into an execution plan. At runtime, the host loads instruction and buffer state, launches an accelerator segment, waits for completion, and then either proceeds to the next accelerator segment or executes a host operation. This continues until the final outputs are produced. The important point is that unsupported operations are not hidden. They are explicit runtime steps, which means their cost and semantic effect can both be observed and measured.

The same dynamic model also supports verification and repeated inference. Verification steps compare runtime tensors with compiler-owned expected tensors when the selected verification mode enables them. Repeated-run sessions allow the simulator backend to preserve reusable state such as static buffer contents and preloaded instruction images. This does not eliminate all setup cost, but it makes the runtime model more realistic for repeated inference and allows overhead reductions to be measured directly.

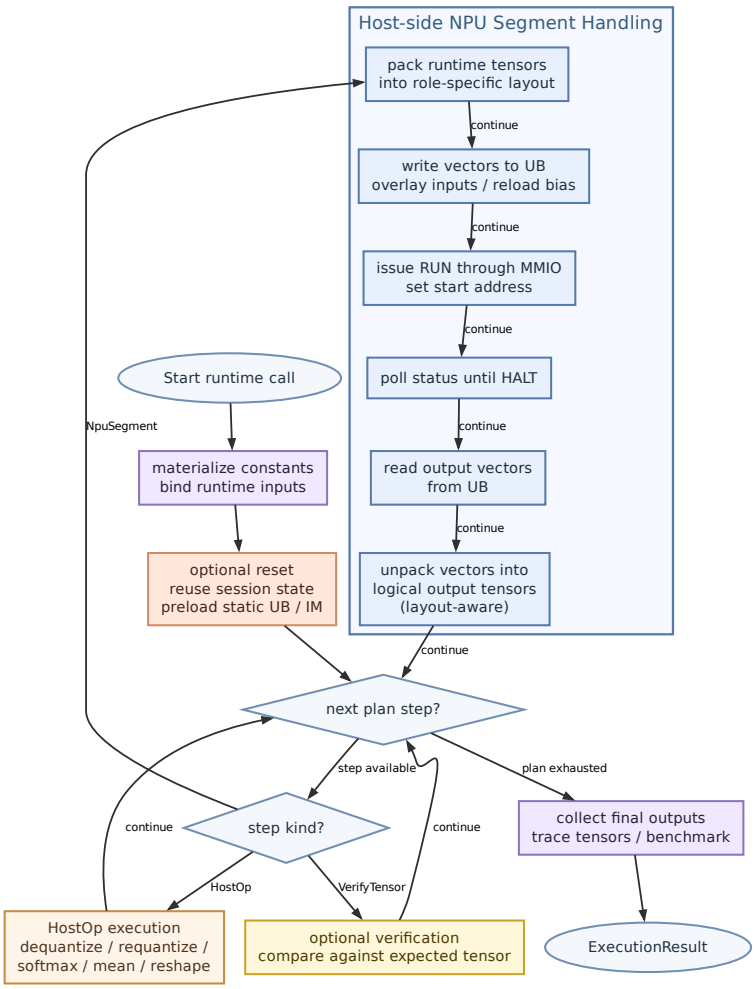

Figure 3.4: Host-side runtime flow for executing a compiled TinyNPU artifact.

Figure 3.4 shows the same runtime loop from the host-side orchestration perspective. For accelerator segments, the runtime first packs logical tensors into the physical layouts expected by the datapath, stages vectors into the unified buffer (shared-SRAM direct path by default, MMIO path as fallback), launches execution, waits for completion, and then reads and unpacks the resulting vectors back into logical tensors. Host-side operations such as requantization, dequantization, softmax, mean, and shape movement appear explicitly in the same loop rather than being hidden behind the accelerator interface. This view is more useful for discussing runtime overhead, because it makes the cost of packing, transfers, polling, and readback visible.

3.3.1 Execution Phases

From the hardware perspective, not every block is active at the same time. The unified buffer, streamer logic, and systolic array form the core pipelined compute path during active matrix-style computation. The post-processing unit participates during the later drain and writeback phase rather than during the main feed phase. This distinction matters for both architectural explanation and performance analysis, because the full accelerator latency is not just the array datapath alone.

From the runtime perspective, these phases are visible as distinct host-side responsibilities. Before the feed phase can begin, the runtime must pack logical tensors into accelerator-facing layouts and write the necessary vectors into the unified buffer. The runtime then issues the launch command and waits while the array consumes the streamed data. After the feed phase, outputs are drained through the post-processing unit, written back in packed form, and finally read and unpacked by the host runtime. This separation is important because it explains why accelerator compute cycles and runtime overhead must be measured separately.

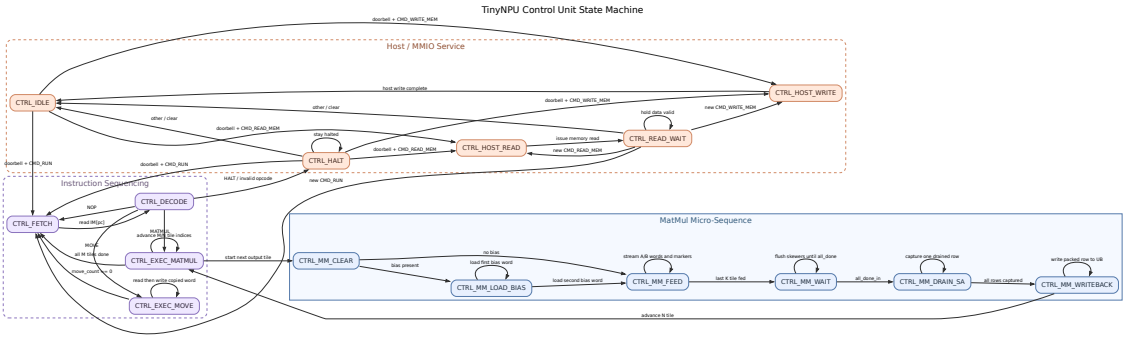

Figure 3.5: Control-unit states tracked by the internal performance counters, grouped by host service, instruction sequencing, and the matrix-multiplication micro-sequence.

Figure 3.5 provides the hardware-side view of those phases. The host-visible states cover MMIO-driven writes, reads, and quiescent behavior. The instruction-sequencing states handle fetch and decode. Once a matrix multiplication begins, the controller

moves through a more detailed micro-sequence including setup, optional bias load, feed, wait, drain, and writeback.

This state machine is not only descriptive; it is also observable. When performance monitoring is enabled, the control unit exposes a total cycle counter together with per-state cycle and entry counters. In other words, the project can record not just how long an accelerator segment took overall, but how that time was distributed across states such as `CTRL_FETCH`, `CTRL_DECODE`, `CTRL_MM_FEED`, `CTRL_MM_WAIT`, and `CTRL_MM_WRITEBACK`. This is useful because it makes the "compute" window interpretable rather than opaque: the measured accelerator run time still contains control and synchronization work in addition to the active datapath feed phase.

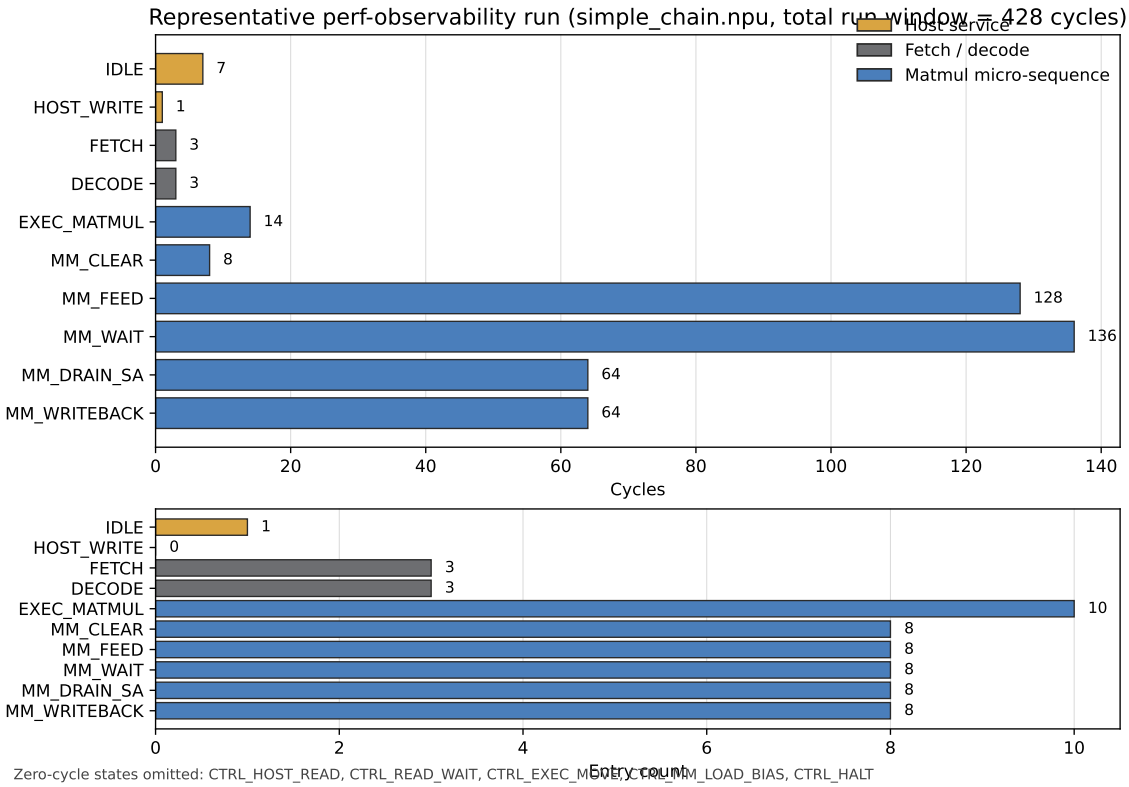

Figure 3.6: Measured per-state cycle and entry breakdown for a representative perf-observability run on `simple_chain.npu`. Zero-cycle states are omitted for readability.

Figure 3.6 turns that observability mechanism into an actual measurement. In this representative run, the dominant states are `CTRL_MM_FEED` and `CTRL_MM_WAIT`, which together account for 264 of the 428 measured cycles. The drain and writeback phases contribute another 128 cycles. By contrast, fetch and decode are only 6 cycles in total, and host-visible service states are negligible inside the measured run window. This is exactly the kind of visibility needed to reason about whether performance is limited by active streaming, array fill and drain behavior, or surrounding control overhead.

3.3.2 Current Scope Boundaries

The current system is deliberately narrower than a general-purpose deployment stack. The mixed-precision helper is meant to produce usable compiler-ready configurations, not to solve the global bit-allocation problem. The compiler handles a bounded set of operators and makes unsupported behavior explicit through host operations rather than pretending to provide transparent full-model lowering. Workload coverage is also still narrow: the current report is grounded in one end-to-end model case and one controlled kernel benchmark. These limitations are acceptable for the present project scope because the main objective is to build and understand a working accelerator system, not to claim a finished production compiler.

4 Experimental Setup and Design

The project was evaluated primarily in simulation, using both host-side execution and RTL-backed cocotb tests. This was necessary because the main goal of the project is not only to confirm functionality, but also to understand how compiler decisions, packing format, and runtime policy affect accelerator behavior before any physical implementation is assumed. Host emulation and RTL simulation share the same execution-plan concept, which made it possible to compare compiler behavior against a golden software model while still collecting hardware-facing evidence.

The experimentation setup contains three layers. First, unit and regression tests validate compiler transformations, execution-plan behavior, memory-planning behavior, and host-op semantics. Second, RTL simulation validates full hardware/software interaction and allows performance counters to be read during accelerator execution. Third, a reproducible Yosys-based synthesis flow provides rough logic and resource estimates for the current top-level design both with and without treating the major memories as abstract macros.

The RTL performance-monitoring path is particularly important for interpreting the accelerator measurements. With performance monitoring enabled, the control unit reports not only a total run-window cycle count, but also per-state cycle and entry counts for its internal control states. This makes it possible to distinguish broad classes of activity such as host service, fetch/decode, feed, wait, drain, and writeback when analyzing where the accelerator is actually spending time during a segment.

The current empirical evidence is intentionally limited and should be described honestly in the report. There is one full end-to-end model case study: a trained MNIST flow that passes through model preparation, compiler-ready conversion, segmented compilation, and RTL-backed validation. There is also one controlled performance case study: a multi-tile matrix multiplication workload large enough to force tiling on the current array configuration. This benchmark is not intended to represent full-model latency by itself; it is a kernel-level workload chosen to isolate the accelerator's core behavior and the effect of integration overhead.

The performance methodology separates different cost categories on purpose. Accelerator compute cycles are taken from RTL performance counters during the run window of a segment. Software-side overhead such as loading buffer images, issuing control transactions, packing and unpacking data, and executing host-only operations is accounted for separately. Several CPU reference models are also applied on the software side, but these are analytic comparison models rather than measurements from a physical reference processor. The purpose of those models is to bound and interpret the accelerator's behavior, not to claim a definitive CPU-versus-NPU benchmark.

For the refreshed Runtime V2 coverage runs in this report, all firmware binaries were built with `-march=rv32imfc -mabi=ilp32 -O3` and `-DTNPU_RUNTIME_V2_DUMP_FINAL_OU`
Runtime validation used `autoverify` against compiler-owned expected tensors when explicit verify ops were not emitted. The simulation command was fixed at `VERILATOR_MAX_TICKS`
with `+maxcycles=250000` to keep runs directly comparable.

5 Comparative Evaluation and Discussion

The current evaluation supports several concrete conclusions, but it should be read with the correct scope. The evidence is sufficient to show that the stack works end to end and that the accelerator core is meaningful. It is not yet broad enough to support sweeping claims about general workload coverage.

First, the end-to-end software and hardware path is real. A trained MNIST model has been prepared for the compiler, compiled through the segmented JIT flow, and validated against RTL. This is important because the central claim of the project is not simply that individual kernels run, but that a model can move from PyTorch-side preparation into a real TinyNPU execution artifact and still preserve host/RTL correctness.

Second, the performance tooling reveals a clear separation between core compute strength and integration cost. On the current multi-tile matrix multiplication benchmark, the RTL run window for the accelerator is only 369 cycles. Depending on the chosen CPU reference model, this translates to a large pure compute speedup range, but the integration-adjusted speedup is much lower because runtime and protocol overhead still matter. This is exactly the kind of conclusion the benchmark path was designed to support: the datapath itself may be strong, while the surrounding software and interface contract still leave room for improvement.

Table 5.1: Controlled multi-tile matrix multiplication benchmark results for the 13×17 by 17×19 workload.

Reference model	CPU replaced cycles	Pure acceleration speedup	Integration-adjusted speedup
Unpipelined scalar	51870	140.57×	8.39×
Ideal issue-1	30628	83.00×	5.20×
Five-stage in-order	43472	117.81×	7.09×

For the same workload, the measured accelerator run window was 369 cycles. Table 5.1 makes the key point visible: the compute core itself is strong, but integration-adjusted speedup is much lower than pure compute speedup because runtime and interface costs are still substantial.

Third, repeated-run runtime work is already justified by measured behavior. On a small repeated-run benchmark, static persistence reduced the modeled NPU-side overhead from 3036 to 1744 units, which corresponds to roughly a 42.6% reduction. On trained MNIST, where the workload is more activation-heavy, the same idea still reduced repeated-run unified-buffer traffic from 17608 words to 14920 words, which is about a 15.3% reduction. These numbers do not claim that the whole problem is solved; they show that repeated-run state reuse is a real and measurable optimization direction.

Table 5.2: Observed effect of repeated-run persistence on two workloads.

Workload	Metric	Baseline	Persistence path	Observed effect
Small repeated-run artifact	NPU overhead cycles	3036	1744	42.6% lower overhead
Trained MNIST	UB words per run	17608	14920	15.3% lower steady-state traffic

Fourth, the Runtime V2 correctness baseline was re-checked after the packed conv-stream RTL fix. The updated matrix is shown in Table 5.3. INT16 and INT8 direct paths now pass end to end, and INT4 also passes when the fixture respects the valid INT4 signed range $[-8, 7]$. The remaining failing INT4 direct fixture is not a datapath failure; it is a test-quality issue because its input tensor contains out-of-range values ($[-14, 14]$).

Table 5.3: Runtime V2 coverage refresh after packed conv-stream RTL update.

Program	Path	Host Im2Col	Stage	Run	Readback	Seg.NPU	Verify
cv32e40p_stream16	conv_stream_same_shape_v2	0	3810	530	3387	13707	OK
cv32e40p_packed8	conv_stream_default_v2	0	7309	98	2931	16267	OK
cv32e40p_i2c_nvu	conv_stream_same_shape_v2	52988	40004	290	9453	55821	OK
cv32e40p_packed4	conv_stream_default_v2	0	2176	82	3199	18386	OK
cv32e40p_packed4	conv_stream_default_v2	0	2176	82	3195	18382	FAIL (@0)

Fifth, the synthesis-oriented characterization confirms that the design is no longer trivial in hardware terms. Under the reproducible Yosys flow, the top-level design synthesized to an estimated 25117 logic cells when the major memories were abstracted and 33644 logic cells when the current memory implementation was kept, with 227 DSP48 blocks in both cases. These numbers are only rough synthesis proxies, not final silicon area, but they are enough to show that TinyNPU is a substantial compute block whose area and timing behavior now deserve serious attention.

The most important interpretation of these results is that the project's center of gravity has shifted. The compiler and runtime exist well enough to run real workloads. The pressing questions are now about workload fit, cycle reduction, memory sizing, and hardware cost, not about whether a compiler path can exist at all.

6 Conclusion and Future Work

TinyNPU has reached the point where it should be treated as a real accelerator platform rather than as a collection of disconnected experiments. The project now contains a usable hardware architecture, a PyTorch-oriented model preparation flow, a segmented compiler/runtime, explicit host-boundary handling, repeated-inference support, verification infrastructure, and benchmark tooling capable of separating compute from overhead. That is a substantial transition from the early stage of simply proving that isolated RTL blocks or simple kernels behaved correctly.

At the same time, the project makes its own limitations visible in a useful way. The current system still pays a noticeable cost for segmentation, state movement, packing, and host-visible boundaries. The mixed-precision path is practical for preparing compiler-ready models, but it is not intended to solve the global bit-allocation problem. Workload coverage is still narrow, and the current performance story is based on one end-to-end model and one controlled kernel benchmark rather than on a broad suite of models.

Those limitations point directly to the next phase of work. The highest-value future work is no longer "make the compiler exist." It is to bring more models onto the stack, reduce avoidable runtime and interface overhead, increase the size of useful accelerator regions, and use real workloads to justify changes in unified-buffer and instruction-memory sizing. On the hardware side, synthesis and timing characterization should be extended beyond rough proxy numbers. On the software side, additional compiler work should be driven by measured workload needs rather than by abstraction for its own sake. That is the most credible path forward for TinyNPU as a research system.

References

[1] Georgios Armeniakos, Francesco Maria Ottavi, Angelo Garofalo, and Luca Benini. Mixed-precision support for RISC-V soft-SIMD extensions, 2024.

[2] Quim Fornt, Georgios Armeniakos, Pasquale Davide Schiavone, and Luca Benini. Mix-GEMM: Mixed-precision RISC-V matrix multiplication using binary segmentation. *IEEE Transactions on Computers*, 2025.

[3] Angelo Garofalo, Francesco Conti, Giuseppe Tagliavini, Davide Rossi, and Luca Benini. XpulpNN: Enabling energy efficient and flexible inference of quantized neural networks on RISC-V based iot end nodes. *IEEE Transactions on Emerging Topics in Computing*, 10(4):1849–1863, 2021.

[4] Hasan Genc, Ameer Haj-Ali, Vivek Ittycheriah, et al. Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration. *Proceedings of the 58th Annual Design Automation Conference (DAC)*, pages 769–774, 2021.

[5] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):12:1–12:25, 2008.

[6] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12, 2017.

[7] Bichen Wu, Yanghan Wang, Peizhao Zhang, Yonglong Tian, Peter Vajda, and Kurt Keutzer. Mixed precision quantization of convnets via differentiable neural architecture search, 2018.

[8] Guangxuan Xiao, Ji Lin, Meryem Seznec, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 38087–38099, 2023.